

Sprawozdanie: Eliminacja elementów zasłoniętych (BSP Tree)

Adam Boros
Kacper Warpechowski

17 maja 2026

1 Cel i zakres zadania

Celem projektu jest stworzenie prostego programu, który używa drzewa BSP do poprawnego rysowania trójkątów w 3D, tak żeby obiekty bliższe zasłaniały te dalsze. Nie korzystamy z Z-bufora — sceny rysowane są w jednolitych kolorach, a dane obiektów wczytywane są z pliku tekstowego.

2 Środowisko uruchomieniowe

Program został napisany w **Pythonie**. Do rysowania używamy `pygame-ce`, a do obliczeń macierzowych `numpy`. Korzystamy z wcześniej napisanej wirtualnej kamery (rzutowanie i sterowanie). Scena jest wczytywana z pliku `scene.txt`.

3 Realizacja graficzna (opis algorytmiczny)

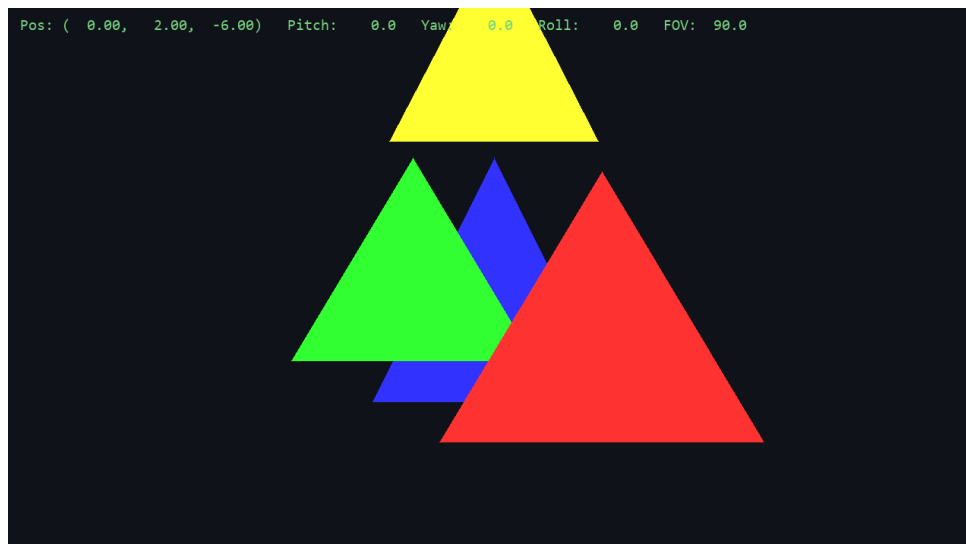
Program czyta plik, w którym każdy wiersz opisuje jeden trójkąt w formacie:

```
x1 y1 z1  x2 y2 z2  x3 y3 z3  R G B
```

Na podstawie tych trójkątów budujemy drzewo BSP. Pierwszy trójkąt służy jako płaszczyzna podziału; pozostałe elementy klasyfikujemy jako przed, za lub (jeśli trzeba) dzielimy na części. Przy rysowaniu przechodzimy drzewo według pozycji kamery i wypisujemy trójkąty od najdalszych do najbliższych (algorytm malarza). Dzięki temu bliższe elementy zasłaniają dalsze.

4 Geometria sceny testowej

Dla testów przygotowaliśmy prostą scenę z czterema trójkątami w różnych kolorach (czerwony, zielony, niebieski i żółty „daszek”) ustawionymi jeden za drugim. To ułatwia sprawdzenie, czy zasłanianie działa poprawnie.



Rysunek 1: Scena 3D widziana frontalnie, tuż po uruchomieniu aplikacji.



Rysunek 2: Układ wygenerowanych trójkątów na siatce współrzędnych - rzut na ułożenie obiektów w głębi sceny.

Zrzuty zamieszczone powyżej (Rysunek 1 oraz Rysunek 2) pełnią formę informacyjną wobec struktury mapy, pokazując tylko konstrukcję oraz ułożenie fizyczne użytych przedmiotów w bezpiecznej odległości i kolejności od Z. Pozwoli to z łatwością ocenić niżej umiejscowione zadania testowe.

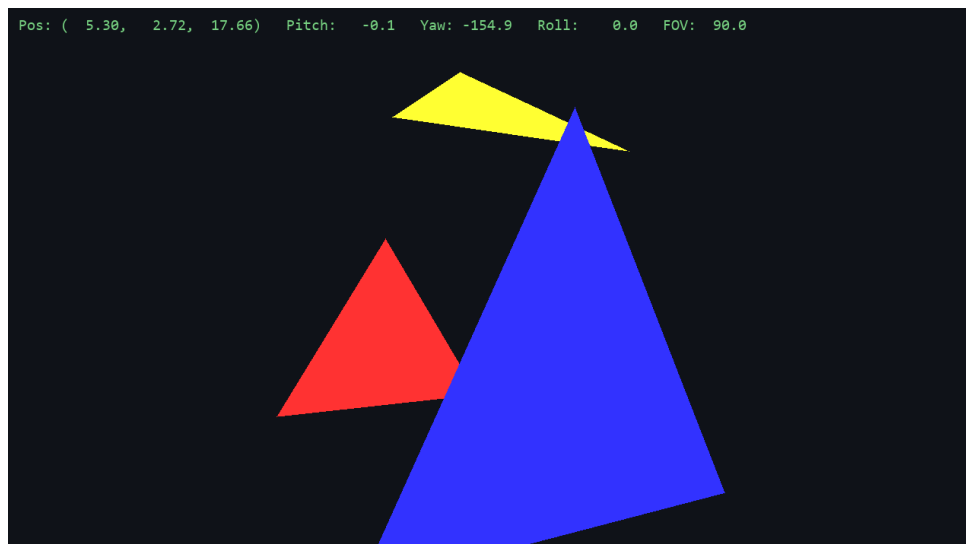
5 Testy działania

Sprawdzaliśmy zachowanie programu głównie przez obracanie i przesuwanie kamery, obserwując, czy obiekty poprawnie zasłaniają się nawzajem.

5.1 Test 1: Przysłonięcia przy rotacji kamery w lewo

Czynność: Obrót kamery w lewo.

Oczekiwany efekt: Niebieski trójkąt, który był dalej, pojawia się z przodu i zasłania elementy, które wcześniej były widoczne.

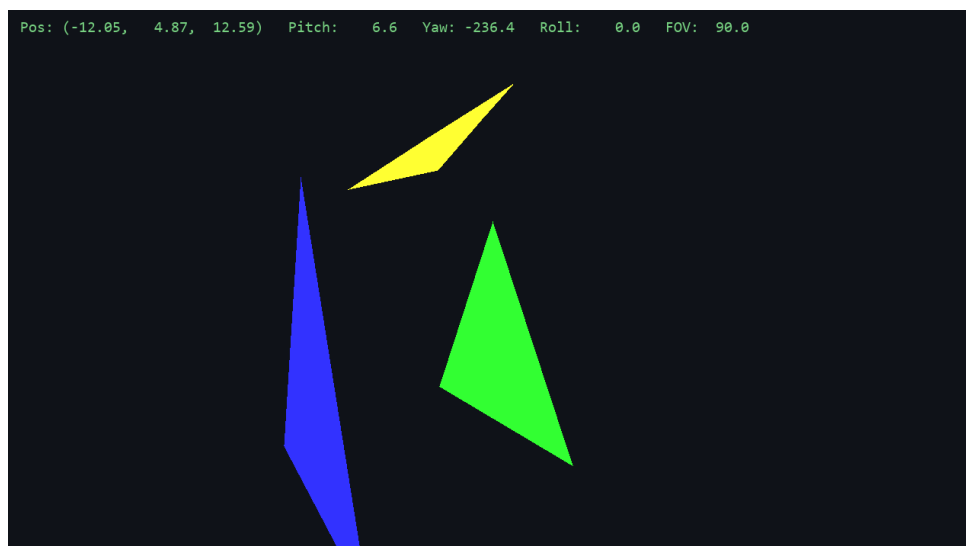


Rysunek 3: Niebieski trójkąt prawidłowo przysłania widok z tyłu, blokując render elementu zielonego w osi wizualnej kamery.

5.2 Test 2: Przysłonięcia przy rotacji kamery w prawo

Czynność: Obrót kamery w prawo.

Oczekiwany efekt: Widok się zmienia tak, że teraz zielony trójkąt może zasłaniać czerwony, zgodnie z nową perspektywą.



Rysunek 4: Zamierzone zakrycie centralnego czerwonego przedmiotu odświeżającą zmianą perspektywy z prawej.

5.3 Test 3: Okluzja elementu ukośnego (pochyłego)

Czynność: Przesunięcie kamery w górę, by spojrzeć na żółty, pochyły trójkąt.

Oczekiwany efekt: Żółty trójkąt zasłania fragmenty trójkątów pod nim, bez powstawania błędnych „wcięć” czy artefaktów.



Rysunek 5: Prawidłowe odcinanie rzutu brył pod pochyłym, skrajnym obiektem.

6 Wnioski końcowe

Drzewo BSP dobrze poradziło sobie z postawionym zadaniem. Dzięki niemu program potrafi posortować trójkąty od najdalszych do najbliższych, więc obiekty położone bliżej poprawnie zasłaniają te dalej. Rozwiązanie zadziałało efektywnie na naszej prostej scenie testowej i pozwoliło uniknąć używania kosztownego bufora Z. To prosty i szybki sposób na usuwanie zasłoniętych elementów, choć w bardzo złożonych scenach mogą być potrzebne dodatkowe poprawki.